

OWL

Z.V. Apanovich

IIS SBRAS, NSU

Web Ontology Language (OWL)

Это семейство языков представления знаний или языков онтологии для описания онтологий или баз знаний.

Языки характеризуются формальной семантикой и сериализацией на основе RDF /XML.

OWL одобрен в World Wide Web Consortium (W3C) и привлек внимание как академического, так и коммерческого сообщества.

Что означает слово «онтология»?

An ontology is an explicit specification of a conceptualization.

—[Tom Gruber](#), *A Translation Approach to Portable Ontology Specifications*¹

Что означает слово «онтология»?

- Данные, описанные онтологией в семействе OWL интерпретируются как набор «индивидуалов» и набор "утверждений свойств", которые связывают этих индивидуалов друг с другом.
- Онтология состоит из набора аксиом, которые устанавливают ограничения на множества индивидуалов (называемых «классами») и типы разрешенных отношений между ними.
- Эти аксиомы обеспечивают семантику, позволяя системам выводить дополнительную информацию на основе явно предоставленных данных.

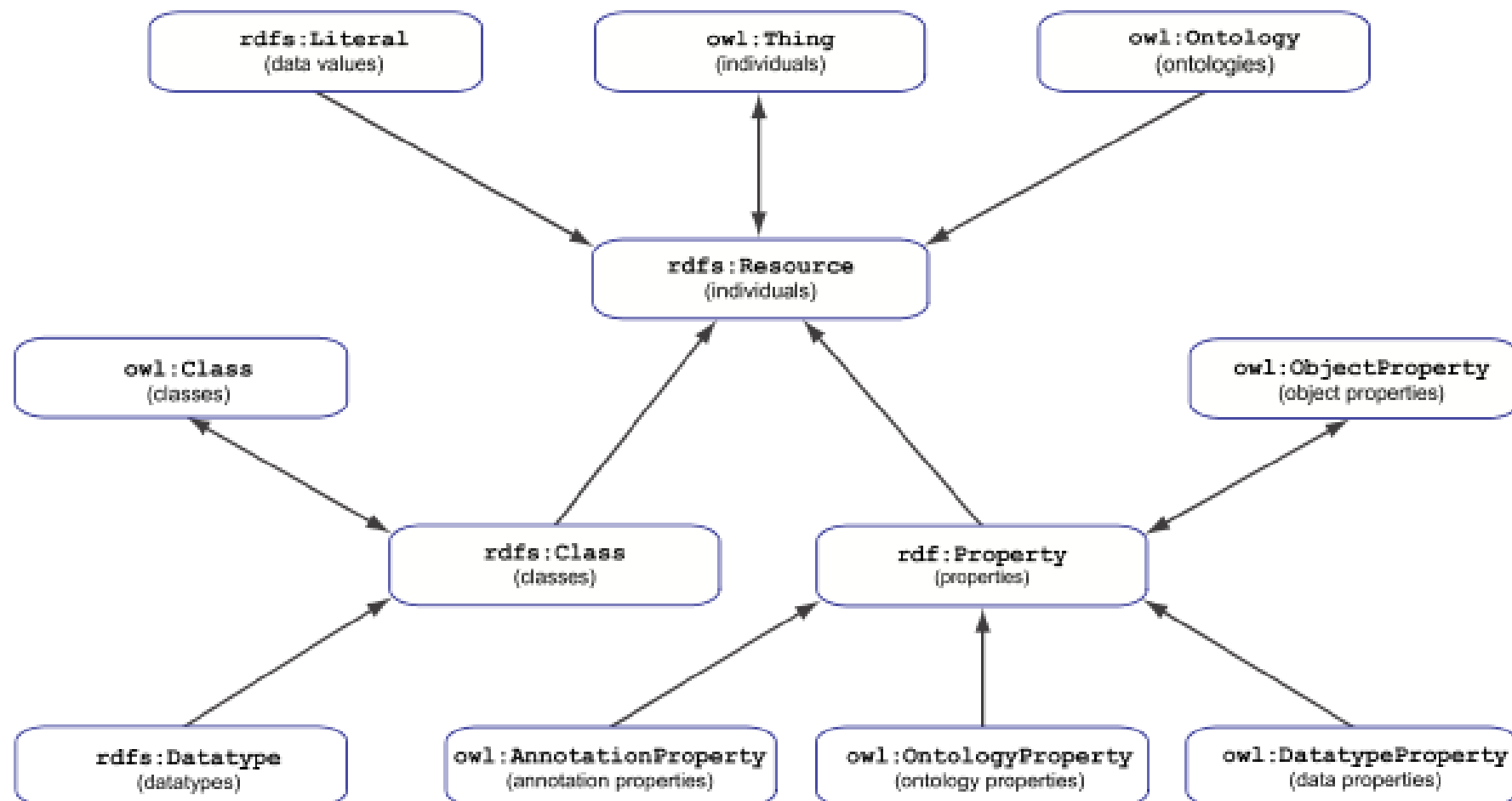
В то время как RDF является современным стандартом для Графов знаний и семантических сетей, стандартным языком описания онтологий является OWL.

OWL основан на семействе формальных представлений знаний, называемых Дескрипционными (Дескриптивными) Логиками (DL).

Сервисы логического вывода онтологий OWL можно использовать для поиска ошибок и несоответствий в графах знаний.

Синтаксически OWL можно рассматривать как расширение RDFS дополнительными терминами, определенными в словаре OWL.

Связи между классами и свойствами RDFS и OWL



OWL

- Не существует единственного стандарта OWL.
- Существуют различные профили OWL-каждый из которых является отдельным подмножеством полного стандартного OWL , которое проще и, в некоторых случаях, гораздо более эффективно вычислительно работает с логическим выводом , чем полный стандарт.
- К счастью, независимо от профиля, с которым вы работаете , все они остаются OWL (например, любая онтология написанная с использованием любого подмножества функций OWL остается законным OWL и должно обрабатываться большинством инструментов OWL).

OWL1

OWL 1 имел под-языки: **OWL FULL, OWL DL, OWL Lite**

OWL FULL неразрешим

OWL DL имеет проблемы с разрешимостью

Даже OWL Lite оказался не очень разрешимым
(EXPTIME-complete)

OWL 2 ввел 3 под-языка, называемых
Профилями, созданные на разные случаи
использования

Профили OWL 2

OWL 2 :

- **EL**: логический вывод за полиномиальное время for schema and data
 - Полезен для онтологий с запутанно связанными классами и свойствами, T-Box
 - $\exists$ + конъюнкция, полиномиальная сложность
- **QL**: быстрые (logspace) ответы на запросы к RDBMs через SQL
 - Полезен для больших наборов данных, уже хранимых в РБД (большие наборы высказываний про индивидуалы A-Box) $\exists$, $\forall$, конъюнкция, дизъюнкция
- **RL**: быстрые (polynomial) ответы на запросы using rule-extended DBs
 - Полезен для больших наборов данных, хранимых в виде триплетов RDF

OWL 2 / Full

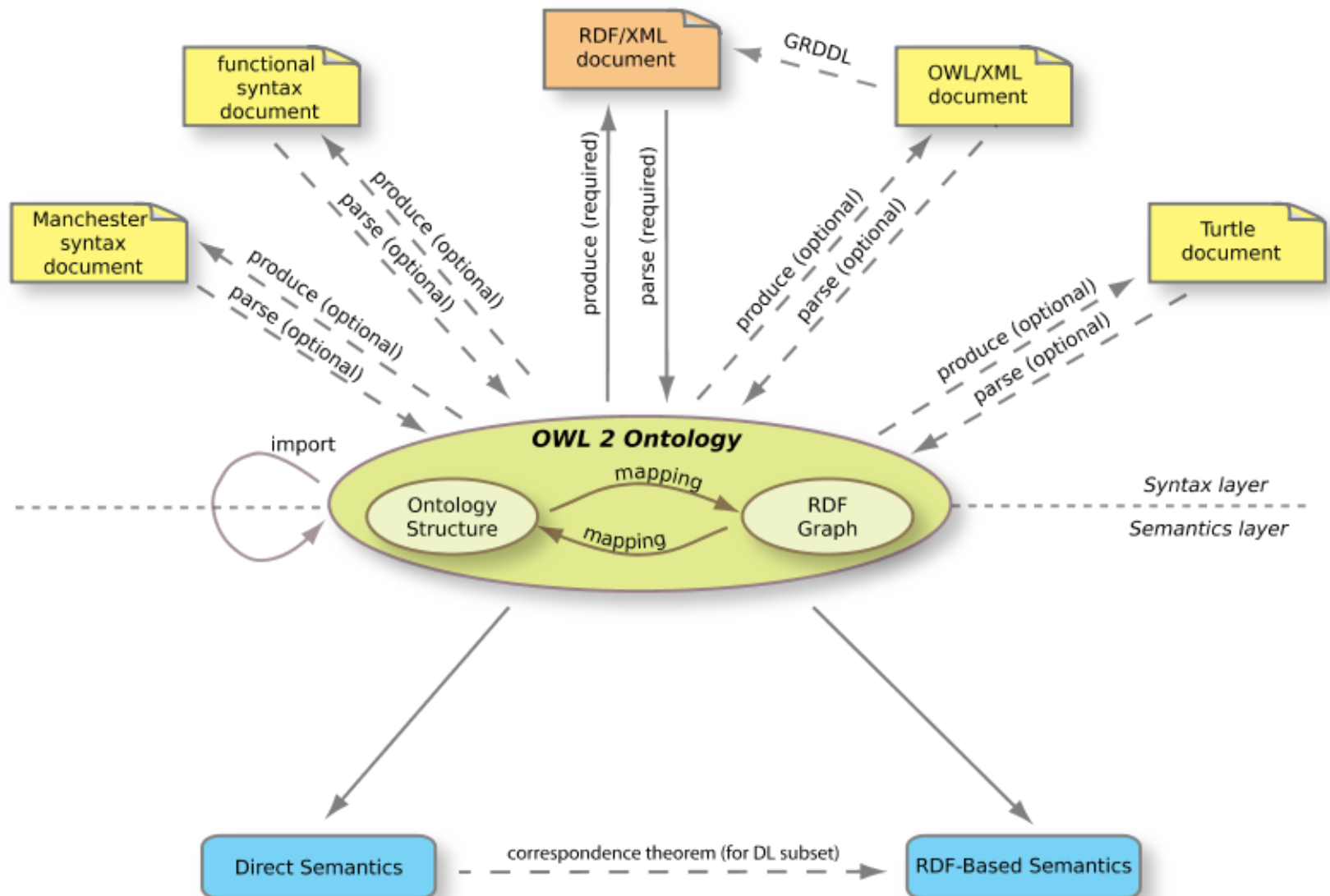
Если вы используете автоматизированные рассуждения, будьте осторожны.

OWL 2/Full определяет ряд правил вывода, которые настолько сложны, что, в лучшем случае, могут работать очень медленно на современных компьютерах.

В худшем случае, можно описать выводы с помощью OWL 2 / Full, которые не могут быть завершены вообще никогда с помощью любого компьютера.

Иначе говоря, логический вывод над OWL 2 / Full неразрешим.

Таким образом, если вы используете автоматизированный логический вывод и вы не хотите работать вечно и/или дать неполные результаты, то вам лучше ограничить себя одним из профилей.



Сервисы логического вывода	Объяснение
Ontology consistency checking	Проверка на наличие противоречий в онтологии
Classification	Вычисление выводимых отношений owl:subClassOf между классами
Realization	Вычисление выводимых отношений rdf:type между индивидуалом и классом
Class satisfiability checking	Проверка возможности наличия экземпляров у класса
Axiom entailment checking	Проверка, может ли аксиома быть выведена из онтологии
Conjunctive query answering	Ответы на запросы относительно онтологии

Эти сервисы реализованы во множестве движко логического вывода таких как FaCT++ , Pellet , HermiT , TrOWL , Knoclude и др.

Пример OWL

- <http://linkeddatatools.com/semantic-web-basics>

OWL example (Декларация пространства имен)

01.<rdf:RDF

02. xmlns:rdf="http://www.w3.org/1999/02/22-
 rdf-syntax-ns#"

03. xmlns:rdfs="http://www.w3.org/2000/01/rdf-
 schema#"

04. xmlns:owl="http://www.w3.org/2002/07/owl
 #"

05. xmlns:dc="http://purl.org/dc/elements/1.1/">

06.

OWL example

07. `<!-- Пример заголовка OWL -->`
08. `<owl:Ontology`
`rdf:about="http://www.linkeddatatools.com/plants">`
09. `<dc:title> Пример онтологии`
`растений</dc:title>`
10. `<dc:description> Пример онтологии`
`</dc:description>`
11. `</owl:Ontology>`
- 12.

OWL example

13. `<!-- Пример определения класса OWL -->`
14. `<owl:Class`
`rdf:about="http://www.linkeddatatools.com/plants#planttype">`
15. `<rdfs:label> Класс растений</rdfs:label>`
16. `<rdfs:comment> Класс типов`
`растений.</rdfs:comment>`
17. `</owl:Class>`
- 18.
19. `</rdf:RDF>`

Классы, подклассы и индивидуалы OWL

- Основной целью онтологии является классификация вещей с точки зрения семантики, или смысла.
- В OWL, это достигается за счет использования классов и подклассов, экземпляры которых в OWL называются индивидуалами.
- Индивидуалы, являющиеся членами данного OWL класса, называются **расширением класса**.

Класс в OWL

- Класс в OWL является классификацией индивидов на группы, которые имеют общие характеристики.
- Если индивид является членом класса, он сообщает читателю-машине, что он подпадает под семантическую классификацию, задаваемую классом OWL.

Пример онтологии OWL (продолжение)

Продолжим пример OWL онтологии, на этот раз с некоторыми добавлениями классов и подклассов.

Определим три класса растений: класс цветущих растений (**flowering plants**) и класс кустарники(**shrubs**), которые оба являются подклассами класса тип_растения **planttype**).

OWL example

```
01.<rdf:RDF
02.  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
03.  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
04.  xmlns:owl="http://www.w3.org/2002/07/owl#"
05.  xmlns:dc="http://purl.org/dc/elements/1.1/"
06.  xmlns:plants="http://www.linkeddatatools.com/plants#">
07.
08.  <!-- OWL Заголовок опущен для краткости-->
09.
10.  <!-- OWL Class Definition - Plant Type -->
11.  <owl:Class rdf:about="http://www.linkeddatatools.com/plants#planttype">
12.
13.    <rdfs:label>Класс растений</rdfs:label>
14.    <rdfs:comment>Класс типов растений </rdfs:comment>
15.
16.  </owl:Class>
```

Пример OWL

```
18. <!--Определение подкласса OWL - Flower -->
19. <owl:Class
    rdf:about="http://www.linkeddatatools.com/plants#flowers">
20.
21.     <!-- Цветы – это подклассификация типов растений -->
22.     <rdfs:subClassOf
        rdf:resource="http://www.linkeddatatools.com/plants#planttype"/>
23.
24.     <rdfs:label>Цветущие растения</rdfs:label>
25.     <rdfs:comment> Цветущие растения, также известные как
        покрытосеменные.</rdfs:comment>
26.
27. </owl:Class>
28.
```

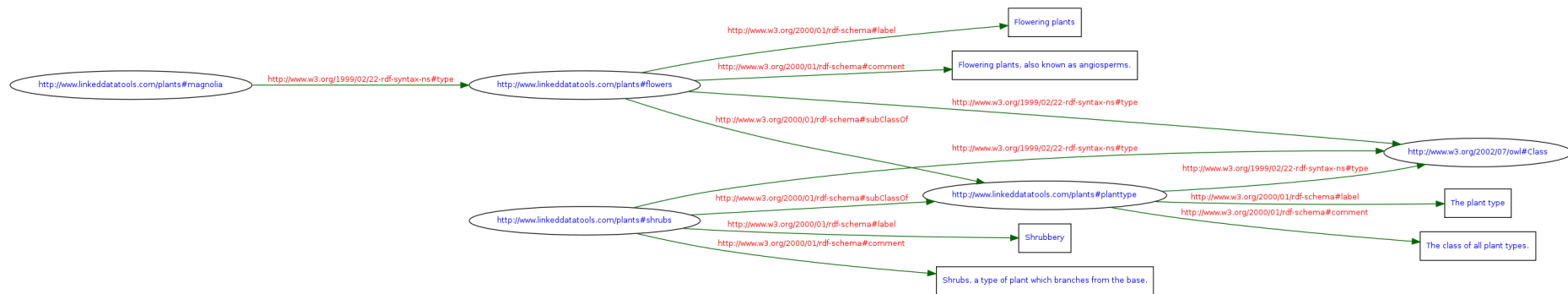
Продолжение

```
29. <!-- OWL Subclass Definition - Shrub -->
30. <owl:Class
    rdf:about="http://www.linkeddatatools.com/plants#shrubs">
31.
32.     <!-- Shrubs is a subclassification of planttype -->
33.     <rdfs:subClassOf
        rdf:resource="http://www.linkeddatatools.com/plants#planttype"/
    >
34.
35.     <rdfs:label>Кустарник</rdfs:label>
36.     <rdfs:comment> rdfs: comment> Кусты, тип растения, ветви
        которого отходят от основания.</rdfs:comment>
37.
38. </owl:Class>
```

Индивидуал

- 40. <!-- Индивидуал (Экземпляр) Пример высказывания RDF -->
- 41. <rdf:Description
rdf:about="http://www.linkeddatatools.com/plants#magnolia">
- 42.
- 43. <!-- Магнолия является экземпляром класса Цветы-->
- 44. <rdf:type
rdf:resource="http://www.linkeddatatools.com/plants#flowers"/>
- 45.
- 46. </rdf:Description>
- 47.
- 48.</rdf:RDF>

Это совершенно легальный граф RDF!

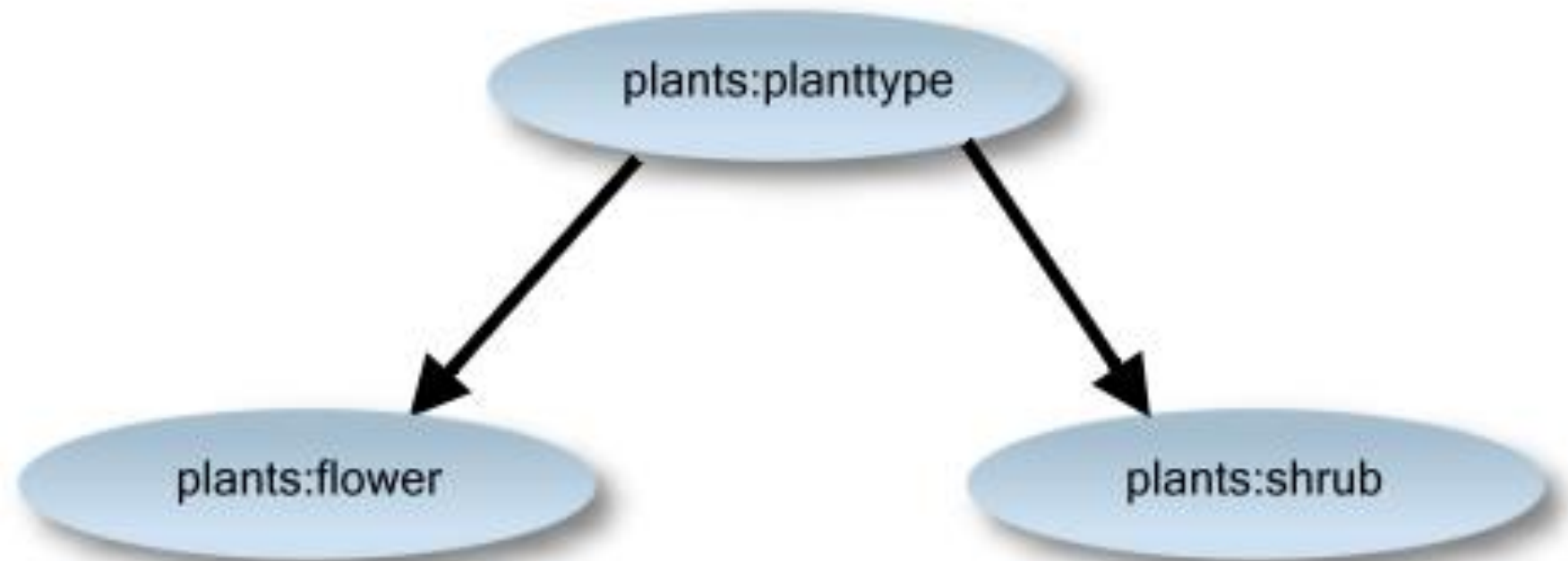


- Таксономия - Иерархия терминов

То, что мы сделали, это определили наши семантические термины, или классы, в иерархии.

В мире semantic web, эта иерархия терминов называется *таксономии*.

Таксономия - это иерархия терминов



An example of a **taxonomy** hierarchy

Магнолия является индивидуалом

Обратите внимание, что не был создан еще один *подкласс* класса цветы, называемый магнолии.

Магнолия описана как *индивидуал* класса цветы.

Почему так сделано?

Магнолия является *членом* цветочной классификации, но это не подклассификация цветов.

OWL Properties

- Индивидуалы в OWL связаны *свойствами*.
Есть два типа *свойств* в OWL:
- Объектные Свойства (**owl: ObjectProperty**) связывают Индивидуалов (экземпляры) двух классов OWL.
- Свойства типов данных (**owl : DatatypeProperty**) связывают Индивидуалов (экземпляры) классов OWL со значениями литералов и типами данных RDF Schema .

Продолжение примера

- Давайте сначала добавим `DatatypeProperty` (которое связывает экземпляр со значением литерала) и добавим имя семейства видов, частью которого является Магнолия.

```
09.
10. <!-- OWL Classes Опущены для краткости-->
11.
12. <!-- Определение свойства family -->
13. <owl:DatatypeProperty rdf:about="http://www.linkeddatatools.com/plants#family"/>
14.
15. <rdf:Description rdf:about="http://www.linkeddatatools.com/plants#magnolia">
16.
17.   <!-- Магнолия является экземпляром класса flowers -->
18.   <rdf:type rdf:resource="http://www.linkeddatatools.com/plants#flowers"/>
19.
20.   <!-- Магнолия является частью семейства магнолиевых (Magnoliaceae)
        Используется элемент свойства, с литеральным значением -->
21.   <plants:family>Magnoliaceae</plants:family>
22.
23. </rdf:Description>
24.
25.</rdf:RDF>
```

Примечание к приведенному выше примеру.

- Свойство 'family' было определено независимо от любого класса, и было присвоено *экземпляру* класса **цветы** (магнолия).
- Другой экземпляр того же класса может не иметь этого свойства.
- Таким образом, обратите внимание, что в OWL, свойства, которые имеют экземпляры не описаны в их типах классов, но в их *экземплярах*.
- В этом случае, вы можете использовать то же свойство «семейство» для экземпляра совершенно другого класса

Пример, ObjectProperty

Наконец, давайте **добавим objectProperty** (которое связывает экземпляр с другим экземпляром).

Например, мы имеем магазин, и мы хотим связать это растение (Magnolia) с другим растением, про которое как владельцу магазина нам известно, что оно является столь же популярным.

Давайте добавим свойство под названием **"similarlyPopularTo"**

```
01.<rdf:RDF
02.  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
03.  xmlns:owl="http://www.w3.org/2002/07/owl#"
04.  xmlns:dc="http://purl.org/dc/elements/1.1/"
05.  xmlns:plants="http://www.linkeddatatools.com/plants#">
06.
07.  <!-- заголовок OWL опущен для краткости-->
08.
09.  <!-- Классы OWL опущены для краткости -->
10.
11.  <!-- Определение свойства family -->
12.  <owl:DatatypeProperty
    rdf:about="http://www.linkeddatatools.com/plants#family"/>
13.
14.  <!-- Определение свойства similarlyPopularTo -->
15.  <owl:ObjectProperty
    rdf:about="http://www.linkeddatatools.com/plants#similarlyPopularTo"/>
16.
```

```
17. <!-- Определение экземпляра класса Орхидея -->
18. <rdf:Description
    rdf:about="http://www.linkeddatatools.com/plants#orchid">
19.
20.     <!-- Орхидея является экземпляром класса flowers -->
21.     <rdf:type
        rdf:resource="http://www.linkeddatatools.com/plants#flowers"/>
22.
23.     <!-- Орхидея является частью семейства Орхидные ('Orchidaceae') -->
24.     <plants:family>Orchidaceae</plants:family>
25.
26.     <!-- Орхидея так же популярна как магнолия-->
27.     <plants:similarlyPopularTo
        rdf:resource="http://www.linkeddatatools.com/plants#magnolia"/>
28.
29. </rdf:Description>
```

```
31. <!-- Определение Магнолии как экземпляра класса-->
32. <rdf:Description
    rdf:about="http://www.linkeddatatools.com/plants#magnolia">
33.
34.     <!-- Магнолия это экземпляр класса flowers -->
35.     <rdf:type
    rdf:resource="http://www.linkeddatatools.com/plants#flowers"/>
36.
37.     <!-- Магнолия это часть семейства магнолиевых ('Magnoliaceae') -->
38.     <plants:family>Magnoliaceae</plants:family>
39.
40.     <!-- Магнолия также популярна как орхидея-->
41.     <plants:similarlyPopularTo
    rdf:resource="http://www.linkeddatatools.com/plants#orchid"/>
42.
43. </rdf:Description>
44.
45.</rdf:RDF>
```

Примечание

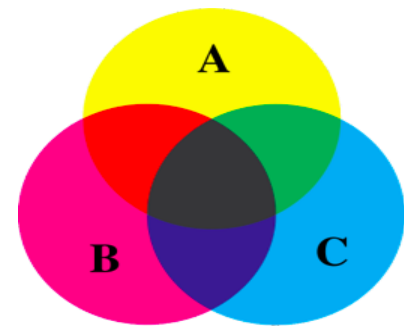
В приведенном выше примере, у нас есть двунаправленные связи между двумя экземплярами того же класса OWL через свойства объекта [similarlyPopularTo](#) (оба Орхидея и Магнолия являются [индивидуалами](#) одного и того же класса).

Однако обратите внимание, что свойства объекта не обязаны быть двухсторонними - они могут быть однонаправленными.

И, что не менее важно, не обязательно должны быть между экземплярами одного и того же класса OWL. Это могут быть совершенно разные классы OWL.

Одной из областей, в которых OWL идет значительно дальше RDFs, это то, что он позволяет строить некоторые довольно сложные, но полезные, отношения между классами.

Некоторые из наиболее распространенных строительных блоков для этого перечислены ниже.



1. Простое объявление класса по идентификатору без указания дополнительных особенностей – это уже знакомый нам способ явного объявления именованного класса.

2) Создание класса при помощи перечисления (`owl:one of`)

В этом случае класс задается явным перечислением входящих в него сущностей .

В синтаксисе RDF/XML такое определение может выглядеть следующим образом:

```
<owl:Class rdf:ID="#BobsChildren">
  <owl:oneOf rdf:parseType="Collection">
    <owl:Thing rdf:about="#Bill"/>
    <owl:Thing rdf:about="#John"/>
    <owl:Thing rdf:about="#Mary"/>
  </owl:oneOf>
</owl:Class>
```

Класс 'BobsChildren' имеет 3 элемента: Bill, John, and Mary

2) Создание класса при помощи перечисления oneOf

```
:BobsChildren owl:equivalentClass [  
  rdf:type owl:Class ;  
  owl:oneOf ( :Bill :John :Mary )  
].
```

3) Создание класса при помощи ограничения на свойства

Ограничение на свойства объявляет некоторый класс, все члены которого удовлетворяют определенному условию.

Условия могут быть двух типов.

1. Все сущности, значение некоторого свойства которых является сущностью определенного класса.
2. Можно накладывать ограничение на количество значений, которые может принимать свойство.

3) Ограничение **allValuesFrom** (RDF/XML).

Все объекты этого свойства ("#eats") являются членами заданного класса ("#NonMeat")

```
<owl:Class rdf:about="#Vegetarian">  
  <owl:equivalentClass>  
    <owl:Restriction>  
      <owl:onProperty rdf:resource="#eats" />  
      <owl:allValuesFrom rdf:resource="#NonMeat" />  
    </owl:Restriction>  
  </owl:equivalentClass>
```

Класс "Вегетарианец" эквивалентен классу вещей, которые едят только не-мясо.

Ограничение **allValuesFrom** ($\forall$) TURTLE

```
:Vegetarian owl:equivalentClass [  
  rdf:type          owl:Restriction ;  
  owl:onProperty   :eats ;  
  owl:allValuesFrom :NonMeat  
].
```

Ограничение someValuesFrom ($\exists$)

... по крайней мере один объект этого свойства является членом определенного класса

```
<owl:Class rdf:about="#Parent">
  <owl:equivalentClass>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#hasChild"/>
      <owl:someValuesFrom rdf:resource="#Person"/>
    </owl:Restriction>
  </owl:Class>
```

Любой экземпляр класса 'Parent' имеет по крайней мере одного ребенка, являющегося персоной (Person)

Ограничение someValuesFrom ($\exists$)

```
:Parent owl:equivalentClass [  
    rdf:type          owl:Restriction ;  
    owl:onProperty   :hasChild ;  
    owl:someValuesFrom :Person  
].
```

Ограничение на количество значений cardinality,min-cardinality,max-cardinality

```
<owl:Class rdf:about="#Automobile">
  <owl:equivalentClass>
    <owl:Restriction>
      <owl:onProperty rdf:resource="#hasWheel" />
      <owl:cardinality
        rdf:datatype="&xsd;nonNegativeInteger">4</owl:cardinality>
    </owl:Restriction>
  </owl:Class>
```

Все автомобили имеют 4 колеса (в отличие от велосипеда).

Ограничение на количество значений
cardinality, min-cardinality, max-cardinality

```
:Automobile owl:equivalentclass [  
  rdf:type      owl:Restriction ;  
  owl:cardinality "4"^^xsd:int ;  
  owl:onProperty :hasWheel  
].
```

Все автомобили имеют 4 колеса (в отличие от велосипеда).

4) Пересечение двух и более классов (`owl:intersectionOf`).

```
<owl:Class>  
  <owl:intersectionOf rdf:parseType="Collection">  
    <owl:Class rdf:about="#Women"/>  
    <owl:Class rdf:about="#Parent"/>  
  </owl:intersectionOf>  
</owl:Class>
```

5) Объединение двух и более классов (owl:unionOf)

```
<owl:Class>  
<owl:unionOf rdf:parseType="Collection">  
<owl:Class rdf:about="#Mother"/>  
<owl:Class rdf:about="#Phather"/>  
</owl:unionOf>  
  
</owl:Class>
```

6) Дополнение описания класса (owl:complementOf).

```
<owl:Class>  
  <owl:complementOf>  
    <owl:Class rdf:about="#Parent"/>  
  </owl:complementOf>  
</owl:Class>
```

Свойства OWL

Свойства в OWL бывают двух основных типов (принадлежат классам):

`owl:DatatypeProperty`

`owl:ObjectProperty`.

И те, и другие являются подклассом свойства вообще:
[`rdf:Property`](#).

Кроме них, существуют и дополнительные, служебные виды свойств, такие как `owl:AnnotationProperty`.

Основные типы свойств

Тип свойства	Использование	Пример	Explanation
DatatypeProperty	...это свойство связывает простыми значениями данных	ex:hasBirthda у	Это свойство связывает с датой
ObjectProperty	...это свойство связывает ресурсы	ex:hasSpouse	Это свойство связывает двух персон

`rdfs:subPropertyOf`

Свойства могут образовывать иерархии точно так же, как классы, при помощи свойства `rdfs:subPropertyOf`.

Например, объявление

```
<owl:ObjectProperty rdf:ID="isMother">  
    <rdfs:subPropertyOf    rdf:resource="#isParent"/>  
</owl:ObjectProperty>
```

rdfs:domain и rdfs:range

Предикаты `rdfs:domain` и `rdfs:range` используются для указания классов, к объектам которых применимо данное свойство, и диапазонов значений свойств.

При обсуждении RDFS мы уже говорили, что указание нескольких `domain` и `range` для свойства интерпретируется так, будто между ними стоит операция логического «И», то есть (в случае `domain`) свойством могут обладать только объекты, относящиеся одновременно ко всем перечисленным классам.

Поэтому если требуется выразить тот факт, что свойством может обладать объединение подклассов — необходимо использовать

`owl:unionOf:`

```
<owl:ObjectProperty rdf:ID="hasBankAccount">
  <rdfs:domain>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <owl:Class rdf:about="#Person"/>
        <owl:Class rdf:about="#Organization"/>
      </owl:unionOf>
    </owl:Class>
  </rdfs:domain>
</owl:ObjectProperty>
```


owl:FunctionalProperty

Много сведений о свойствах можно выразить путем их отнесения к специальным классам OWL.

Так, отнесение свойства к классу **owl:FunctionalProperty** является «глобальным ограничением»: оно утверждает, что каждый носитель этого свойства может иметь одно и только одно значение.

owl:FunctionalProperty является подклассом rdf:Property.

Для того, чтобы объявить свойство функциональным, нужно отнести его к этому типу, наряду с основным имеющимся у него типом – ObjectProperty или DatatypeProperty:

owl:FunctionalProperty Пример

```
<owl:ObjectProperty rdf:ID="husband">  
  <rdf:type  
rdf:resource="&owl;FunctionalProperty" />  
  <rdfs:domain rdf:resource="#Woman" />  
  <rdfs:range rdf:resource="#Man" />  
</owl:ObjectProperty>
```

У каждой женщины может быть только один муж

owl:InverseFunctionalProperty

Могут существовать и обратные функциональные свойства, owl:InverseFunctionalProperty – в этом случае на каждый объект может указывать только одна связь данного типа.

Например, выражение

```
<owl:InverseFunctionalProperty  
  rdf:ID="biologicalMotherOf">  
  <rdfs:domain rdf:resource="#Woman"/>  
  <rdfs:range rdf:resource="#Human"/>  
</owl:InverseFunctionalProperty>
```

утверждает, что у каждого человека есть только одна биологическая мать, при этом свойство «biologicalMotherOf» указывает от женщины на человека.

Отношения между свойствами owl:equivalentProperties , owl:inverseOf

Свойства могут находиться в определенных отношениях между собой.

Выражение **owl:equivalentProperties** утверждает, что два свойства, обладающие разными идентификаторами, имеют один и тот же смысл.

Выражение owl:inverseOf утверждает, что одно свойство является обратным другому:

```
<owl:ObjectProperty rdf:ID="hasChild">  
<owl:inverseOf rdf:resource="#hasParent"/>  
</owl:ObjectProperty>
```

Типы свойств: owl:TransitiveProperty

Свойства могут обладать определенными логическими характеристиками.

Свойство может быть транзитивным, owl:TransitiveProperty – это означает, что если А связано неким свойством с В, а В с С, то А связано этим же свойством с С.

Например, если Анна выше чем Иван, а Иван выше чем Игорь, то Анна выше чем Игорь.

Для этого и пригодится определение транзитивного свойства:

```
<owl:ObjectProperty rdf:ID="isTaller">  
<rdf:type rdf:resource="&owl;TransitiveProperty"/>  
<rdfs:domain rdf:resource="#Person"/>  
<rdfs:range rdf:resource="#Person"/>  
</owl:ObjectProperty>
```

Впрочем, owl:TransitiveProperty является подклассом owl:ObjectProperty,

поэтому объявлять ObjectProperty в явном виде в данном случае не обязательно

Типы свойств: owl:SymmetricProperty

Свойство может быть симметричным: это означает, что если А имеет некое отношение с В, то и В имеет такую же отношение с А.

Например, для отношения «является супругом» запись будет такой:

```
<owl:SymmetricProperty rdf:ID="hasSpouse">  
  <rdfs:domain rdf:resource="#Human"/>  
  <rdfs:range rdf:resource="#Human"/>  
</owl:SymmetricProperty>
```

Kind of Property	Used to say...	Example	Explanation
TransitiveProperty	... если это свойство связывает А с В и В с С, то оно также связывает А с С	ex:tallerThan	If Ann is taller than Bob, and Bob is taller than Chuck, then Ann is taller than Chuck
SymmetricProperty	...если свойство связывает А с В, то оно всегда связывает В с А	ex:hasSpouse	If Ann is Bob's spouse, then Bob is Ann's spouse too
AsymmetricProperty	...если свойство связывает А с В, то оно НИКОГДА не связывает В с А.	ex:tallerThan	If Ann is taller than Bob, then Bob <i>can't be</i> taller than Ann
ReflexiveProperty	..это свойство всегда связывает что-то с самим собой.	ex:livesWith	Everybody lives with themselves
IrreflexiveProperty	...это свойство никогда не связывает что-то с самим собой..	ex:hasSpouse	Nobody is their own spouse
FunctionalProperty	..это свойство связывает только с одной сущностью	ex:hasBirthday	You only have one birthday
InverseFunctionalProperty	..субъект этого свойства однозначно идентифицируется значением этого свойства.	ex:hasDLNumber	I am the only person with my drivers license number

Linked Open Vocabularies (LOV)

Windows Internet Explorer window showing the Linked Open Vocabularies (LOV) search interface.

Address bar: <http://lov.okfn.org/dataset/lov/search/#s=Publication>

Page Title: (LOV) Linked Open Vocabularies - Windows Internet Explorer

Page Content:



Linked Open Vocabularies (LOV)

developed by Pierre-Yves Vandenbussche

The "LOV Search" Features gives you the possibility to search for an existing element (property, class or vocabulary) in the Linked Open Vocabularies Catalogue.
[LOV Aggregator endpoint](#) and [metrics](#) about the use of vocabularies in the Semantic Web are used to bring you some relevant results.

 Endpoint

Search bar:

Filter by Domain

- City (32)
- Data & Systems (3)
- General (19)
- Library (72)
- Market (4)
- Media (31)
- Metadata (18)
- Science (39)

Filter by Type

- rdfs:Class (92)
- rdf:Property (137)
- voaf:Vocabulary (19)
- Other (151)

Filter by Vocabulary (51)

- ebucore (23)
- opus (19)
- rdag1 (14)
- uniprot (12)
- etc (14)

340 results in 51 vocabularies

Vocabulary	Type	Score
foaf:publications	(owl:ObjectProperty)	score:0.644
rdfs:label Publications @en rdfs:label publications dce:title Publications @en rdfs:comment "A link to the publications of this person." T.....tes an agent to its publications. NOTE: foaf define.....pany) can also have publications (with no specific ... rdfs:comment A link to the publications of this person. dce:description "A link to the publications of this person." T.....tes an agent to its publications. NOTE: foaf define.....pany) can also have publications (with no specific ... @en		
swpo:Publication	(owl:Class)	score:0.606
rdfs:label Publication @en dce:title Publication @en rdfs:comment Publications are both individua... dce:description Publications are both individua... @en		
gndo:publication	(owl:DatatypeProperty)	score:0.571
rdfs:label Publication @en		
pdo:Publication	(owl:Class)	score:0.530
rdfs:label Publication rdfs:comment ... the structure of a publication		
vdpp:Publication	(owl:Class)	score:0.530
rdfs:label Publication rdfs:comment The result of the publication is a URI where the ...		
cogs:Publication	(owl:Class)	score:0.530
rdfs:label Publications		